

Assignment 1 – DBMS vs RDBMS

Scenario:

A small grocery shop currently stores customer details in Excel files. The owner wants to manage customers, products, and sales efficiently while avoiding duplicate data.

Task:

1. Explain why an RDBMS is more suitable than a simple DBMS.

A plain DBMS (or a flat file/Excel system) stores data as independent files with no enforced structure between them, so the same customer or product details often get typed out again in every sheet — leading to duplication, inconsistency, and wasted storage. An RDBMS (Relational Database Management System) organizes data into related tables (Customers, Products, Sales) that are linked using keys instead of repeating information.

- Eliminates redundancy — customer details are stored once and referenced by ID in the Sales table.
- Maintains data integrity — relationships (Primary Key/Foreign Key) prevent orphan or inconsistent records.
- Supports powerful querying — SQL allows filtering, joining, and reporting across Customers, Products, and Sales instantly.
- Enforces constraints — data types, uniqueness, and NOT NULL rules stop bad data from being entered, unlike Excel.
- Scales better and supports multiple concurrent users, which a spreadsheet cannot handle safely.

2. List three advantages of using MySQL.

- Free and open-source — no licensing cost, making it ideal for a small business, with a large community for support.
- Fast and reliable performance, even with large volumes of transactional data such as daily sales records.
- Strong security features — user accounts, privileges, and access control protect sensitive customer data, something Excel files can't offer.

Assignment 2 – Tables, Rows, Columns

Scenario:

A college wants to create a **Student** table.

Task:

1. Design a Student table with at least **6 columns**.

```

11 • insert into student (Stud_id,Stud_firstname,Stud_lastname,DOB,Gender,stud_class,stud_email) values
12 (1, 'Arun', 'Kumar', '2005-03-12', 'Male', 'BSc-CS-I', 'arun.k@example.com'),
13 (2, 'Divya', 'Raj', '2005-07-21', 'Female', 'BSc-CS-I', 'divya.r@example.com'),
14 (3, 'Karthik', 'Suresh', '2004-11-05', 'Male', 'BSc-CS-II', 'karthik.s@example.com'),
15 (4, 'Priya', 'Menon', '2005-01-30', 'Female', 'BSc-CS-I', 'priya.m@example.com'),
16 (5, 'Vishal', 'Nair', '2004-09-17', 'Male', 'BSc-CS-II', 'vishal.n@example.com');
17 • show tables from student;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Tables_in_student

student

2. Insert 5 sample student records.

```

11 • insert into student (Stud_id,Stud_firstname,Stud_lastname,DOB,Gender,stud_class,stud_email) values
12 (1, 'Arun', 'Kumar', '2005-03-12', 'Male', 'BSc-CS-I', 'arun.k@example.com'),
13 (2, 'Divya', 'Raj', '2005-07-21', 'Female', 'BSc-CS-I', 'divya.r@example.com'),
14 (3, 'Karthik', 'Suresh', '2004-11-05', 'Male', 'BSc-CS-II', 'karthik.s@example.com'),
15 (4, 'Priya', 'Menon', '2005-01-30', 'Female', 'BSc-CS-I', 'priya.m@example.com'),
16 (5, 'Vishal', 'Nair', '2004-09-17', 'Male', 'BSc-CS-II', 'vishal.n@example.com');
17 • show tables from student;
18 • select * from student;
19

```

Limit to 1000 rows

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Stud_id	Stud_Firstname	Stud_lastname	DOB	Gender	Stud_class	Stud_Email
1	Arun	Kumar	2005-03-12	Male	BSc-CS-I	arun.k@example.com
2	Divya	Raj	2005-07-21	Female	BSc-CS-I	divya.r@example.com
3	Karthik	Suresh	2004-11-05	Male	BSc-CS-II	karthik.s@example.com
4	Priya	Menon	2005-01-30	Female	BSc-CS-I	priya.m@example.com
5	Vishal	Nair	2004-09-17	Male	BSc-CS-II	vishal.n@example.com

3. Identify the rows and columns.

7 columns -

Stud_id,Stud_firstname,Stud_lastname,DOB,Gender,stud_class,stud_email

5 Rows

- (1, 'Arun', 'Kumar', '2005-03-12', 'Male', 'BSc-CS-I', 'arun.k@example.com'),
- (2, 'Divya', 'Raj', '2005-07-21', 'Female', 'BSc-CS-I', 'divya.r@example.com'),
- (3, 'Karthik', 'Suresh', '2004-11-05', 'Male', 'BSc-CS-II', 'karthik.s@example.com'),
- (4, 'Priya', 'Menon', '2005-01-30', 'Female', 'BSc-CS-I', 'priya.m@example.com'),
- (5, 'Vishal', 'Nair', '2004-09-17', 'Male', 'BSc-CS-II', 'vishal.n@example.com')

Assignment 3 – Primary Key

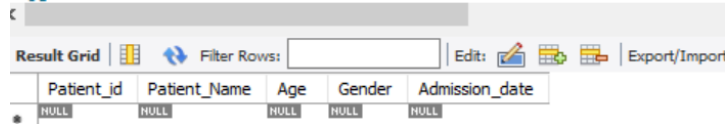
Scenario:

A hospital stores patient information.

Task:

1. Design a **Patient** table.

```
19 • create database patient;
20 • use patient;
21 • create table hos_patient(
22     Patient_id Int primary key,
23     Patient_Name Varchar(20) NOT Null,
24     Age Int,
25     Gender Varchar(15),
26     Admission_date DATE);
27 • select * from hos_patient;
```



2. Select an appropriate Primary Key.

I chose patient id as primary key

3. Explain why it should be unique.

Primary key is patient id because one patient will have one id which represents one individual person. No two person can exist with same id. So Patient id is unique and hence chose for primary key

Assignment 4 – Foreign Key

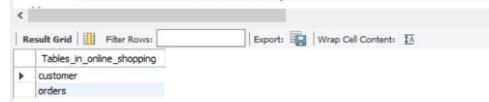
Scenario:

An online shopping website stores customer and order information.

Task:

1. Create **Customer** and **Orders** tables.

```
36 Orderid int primary key,
37 Orderdate date,
38 Amount decimal(10,0),
39 customerid int,
40 Foreign key (customerid) references customer(customerid));
41 • select * from customer;
42 • select * from orders;
43 • show tables from online_shopping;
```



2. Connect them using a Foreign Key. 3.Explain the relationship.

CustomerID in the Orders table is a Foreign Key that references CustomerID (the Primary Key) in the Customer table. This creates a one-to-many relationship: one customer can place many orders, but each order belongs to exactly one customer.

Assignment 5 – ER Diagram

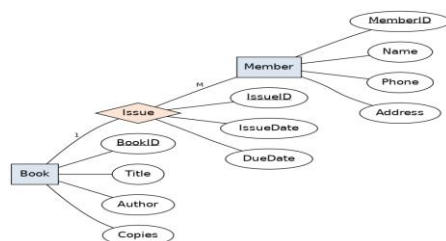
Scenario:

A library stores books and members.

Task:

Draw an ER Diagram containing:

- Book
- Member
- Issue



Include:

- Attributes
- Relationships
- Primary Keys

Entity	Attributes (PK underlined)	Relationship
Book	<u>BookID</u> (PK), Title, Author, Copies	A Book can be issued many times (1..N Issue records)
Member	<u>MemberID</u> (PK), Name, Phone, Address	A Member can hold many Issue records
Issue	<u>IssueID</u> (PK), IssueDate, DueDate, <u>BookID</u> (FK), <u>MemberID</u> (FK)	Links one Book to one Member for a borrowing event

Assignment 6 – Normalization (1NF)

Scenario:

The following table stores multiple phone numbers in one column.

StudentID	Name	Phone Numbers
101	Arun	9876543210, 9876501234

Task:

Convert the table into **First Normal Form (1NF)**.

StudentID	Name	Phone Number
101	Arun	9876543210
101	Arun	9876501234

Assignment 7 – Normalization (2NF)

Scenario:

A Student Course table stores:

StudentID, StudentName, CourseID, CourseName

Task:

Split the table into appropriate tables to achieve **Second Normal Form (2NF)**.

CREATE TABLE Students (

StudentID INT PRIMARY KEY,

StudentName VARCHAR(100)

);

CREATE TABLE Course (

CourseID INT PRIMARY KEY,

CourseName VARCHAR(100)

);

CREATE TABLE Enrollment (

StudentID INT,

CourseID INT,

PRIMARY KEY (StudentID, CourseID),

FOREIGN KEY (StudentID) REFERENCES Students(StudentID),

```
FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
```

Assignment 8 – Normalization (3NF)

Scenario:

An Employee table stores:

EmployeeID, EmployeeName, DepartmentID, DepartmentName

Task:

Normalize the table into **Third Normal Form (3NF)**.

```
CREATE TABLE Department (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(100)
);

CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(100),
    DepartmentID INT,
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);
```

DepartmentID	DepartmentName
D1	HR
D2	Finance

EmployeeID	EmployeeName	DepartmentID
1	John	D1
2	Sarah	D2

Assignment 9 – SELECT Statement

Scenario:

A company has an Employee table.

Task:

Write SQL queries to:

- Display all records.

EmployeeID	EmployeeName	Salary	DepartmentName	City	Designation	Email	mobile
101	John Smith	65000	HR	Chennai	HR Executive	john.smith@example.com	9876543210
102	Alice Johnson	72000	Finance	Bengaluru	Accountant	alice.johnson@example.com	9876543211
103	Michael Brown	85000	IT	Hyderabad	Software Engineer	michael.brown@example.com	9876543212
104	Sophia Davis	95000	IT	Pune	Senior Software Engineer	sophia.davis@example.com	9876543213
105	David Wilson	70000	Marketing	Mumbai	Marketing Executive	david.wilson@example.com	9876543214
106	Emma Taylor	60000	Sales	Delhi	Sales Executive	emma.taylor@example.com	9876543215
107	Daniel Thomas	110000	Finance	Chennai	Finance Manager	daniel.thomas@example.com	9876543216
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Display only Employee Name and Salary.

```

103 • select * from employee1;
104 • select EmployeeName,salary from employee1 where employeeName='John Smith';
105

```

EmployeeName	salary
John Smith	65000

- Display all Department Names.

```

105 • select distinct departmentName from employee1;
106
107
108
109

```

departmentName
HR
Finance
IT
Marketing
Sales

Assignment 10 – WHERE Clause

Scenario:

A school stores marks of students.

```

104 • select EmployeeName,salary from employee1 where employeeName='John smith';
105 • select distinct departmentName from employee1;
106 • select * from student1;
107
108

```

StudentID	Name	Marks	City	Grade	Fees
101	Aarav	92	Chennai	A	50000
102	Diya	85	Bengaluru	B	48000
103	Rohan	76	Hyderabad	C	45000
104	Meera	95	Mumbai	A	52000
105	Karthik	68	Coimbatore	D	40000
106	Sneha	81	Pune	B	47000
107	Rahul	89	Delhi	A	50000

Task:

Write SQL queries to display:

- Students scoring above 80 marks.

```

107 • select name from student1 where marks>80;

```

name
Aarav
Diya
Meera
Sneha
Rahul

- Students from Chennai.

```

108 • select name from student1 where city='chennai';

```

name
Aarav

- Students studying in Grade 10.

```

121 • select * from student1;
122 • select name from student1 where class=10;

```

name
Rohan
Karthik
Rahul

Assignment 11 – ORDER BY

Scenario:

An HR department wants reports.

Task:

Write SQL queries to:

- Sort employees by salary (Highest to Lowest).

```
122 • select name from student1 where class=10;
123 • select employeeename, salary from employee1 order by salary desc;
```

<

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	employeeename	salary
▶	Daniel Thomas	110000
	Sophia Davis	95000
	Michael Brown	85000
	Alice Johnson	72000
	David Wilson	70000
	John Smith	65000
	Emma Taylor	60000

- Sort employees alphabetically by name.

```
124 • select employeeename from employee1 order by employeeename asc;
```

<

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	employeeename
▶	Alice Johnson
	Daniel Thomas
	David Wilson
	Emma Taylor
	John Smith
	Michael Brown
	Sophia Davis

- Sort students by marks (Lowest to Highest).

```
125 • select name, marks from student1 order by marks asc;
```

<

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	name	marks
▶	Karthik	68
	Rohan	76
	Sneha	81
	Diya	85
	Rahul	89
	Aarav	92

Assignment 12 – DISTINCT

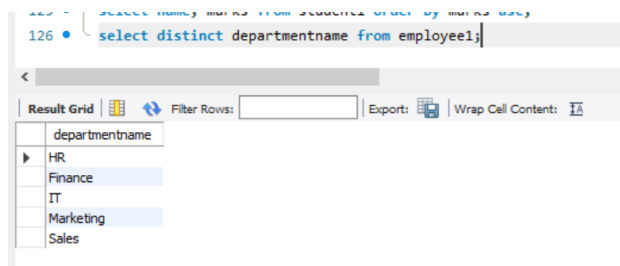
Scenario:

A company has many employees working in different departments.

Task:

Write SQL queries to display:

- Unique Department Names.

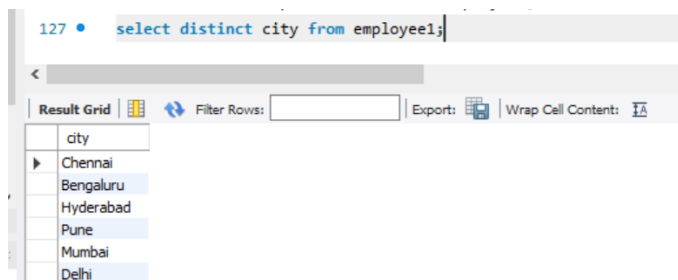


126 • `select distinct departmentname from employee1;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

departmentname
HR
Finance
IT
Marketing
Sales

- Unique Cities.

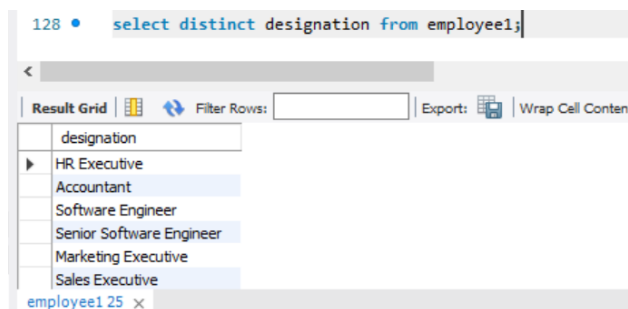


127 • `select distinct city from employee1;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

city
Chennai
Bengaluru
Hyderabad
Pune
Mumbai
Delhi

- Unique Designations.



128 • `select distinct designation from employee1;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

designation
HR Executive
Accountant
Software Engineer
Senior Software Engineer
Marketing Executive
Sales Executive

employee1 25 x

Assignment 13 – SQL Comments

Scenario:

You are writing SQL scripts for another developer.

Task:

Write SQL statements using:

- Single-line comments

```
-- This is a single-line comment: fetch all active employees
SELECT * FROM Employee;
```

- Multi-line comments

```
/* This is a multi-line comment.
   It explains that the query below
   returns employees earning above 50,000 */
```

- Explain why comments are useful.

Comments explain the purpose or logic of a query without affecting execution. They help other developers quickly understand what a script does, make debugging and maintenance easier, and are especially valuable in long scripts where the intent of a query isn't obvious at a glance.

Assignment 14 – SQL Data Types

Scenario:

A bank wants to create an Account table.

Task:

Choose appropriate SQL data types for:

- Account Number
- Customer Name
- Balance
- Opening Date
- Mobile Number
- Email

Explain your choices.

Field	Suggested Data Type	Reason
Account Number	VARCHAR(20)	May include leading zeros or bank-specific formatting; treated as an identifier, not used in arithmetic.
Customer Name	VARCHAR(100)	Variable-length text of reasonable maximum length.
Balance	DECIMAL(12,2)	Exact precision is required for money — floating point types can introduce rounding errors.

Opening Date	DATE	Stores calendar date only, enabling date arithmetic and sorting.
Mobile Number	VARCHAR(15)	Stored as text since it may include a leading '+' or country code and is never used in calculations.
Email	VARCHAR(100)	Variable-length text sufficient for standard email address lengths.

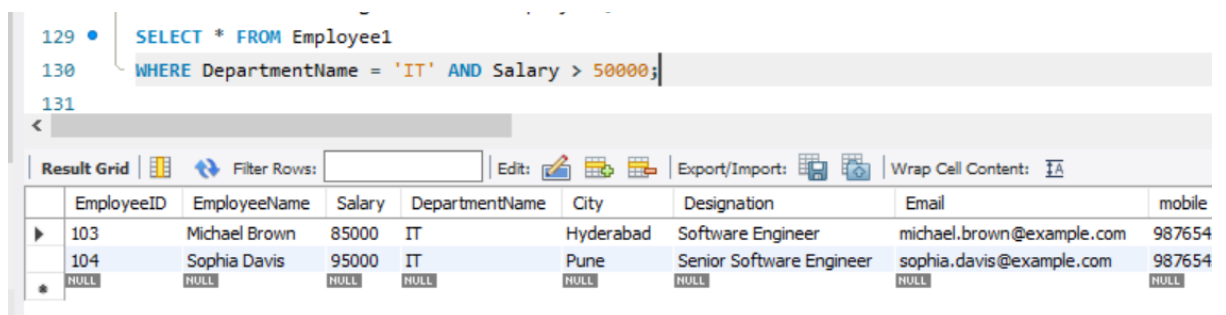
Assignment 15 – AND Operator

Scenario:

An HR manager wants employees from the IT department earning more than ₹50,000.

Task:

Write an SQL query using **AND**.



```
129 • SELECT * FROM Employee1
130 WHERE DepartmentName = 'IT' AND Salary > 50000;
131
```

EmployeeID	EmployeeName	Salary	DepartmentName	City	Designation	Email	mobile
103	Michael Brown	85000	IT	Hyderabad	Software Engineer	michael.brown@example.com	987654
104	Sophia Davis	95000	IT	Pune	Senior Software Engineer	sophia.davis@example.com	987654

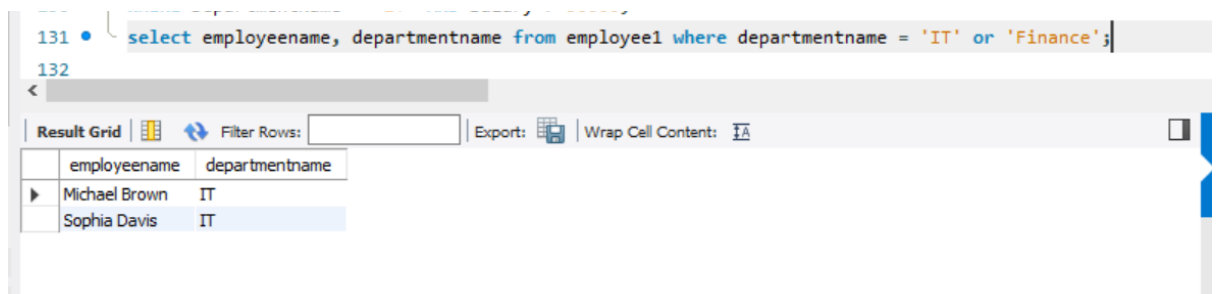
Assignment 16 – OR Operator

Scenario:

A company wants employees working in either HR or Finance.

Task:

Write an SQL query using **OR**.



```
131 • select employeename, departmentname from employee1 where departmentname = 'IT' or 'Finance';
132
```

employeename	departmentname
Michael Brown	IT
Sophia Davis	IT

Assignment 17 – NOT Operator

Scenario:

A company wants all employees except those working in Sales.

Task:

Write an SQL query using **NOT**.

```
131 • select employeeename, departmentname from employee1 where departmentname = 'IT' or 'Finance';
132 • select employeeename,departmentname from employee1 where not departmentname = 'sales';
133
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

employeeename	departmentname
John Smith	HR
Alice Johnson	Finance
Michael Brown	IT
Sophia Davis	IT
David Wilson	Marketing
Daniel Thomas	Finance

Assignment 18 – BETWEEN

Scenario:

A teacher wants students scoring between 60 and 80 marks.

Task:

Write an SQL query using **BETWEEN**.

```
133 • select name, marks from student1 where marks between 60 and 80;
134
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

name	marks
Rohan	76
Karthik	68

Assignment 19 – IN Operator

Scenario:

A company wants employees from Chennai, Bangalore, or Hyderabad.

Task:

Write an SQL query using **IN**.

```

134 • select employeeName from employee1 where city in ('Chennai','Bangalore','Hyderabad');
135

```

Result Grid

employeeName
John Smith
Michael Brown
Daniel Thomas

Assignment 20 – LIKE Operator

Scenario:

A customer database contains thousands of customer names.

Task:

Write SQL queries to display:

- Names starting with 'A'

```

136 • SELECT * FROM employee1 WHERE employeeName LIKE 'A%';
137

```

Result Grid

EmployeeID	EmployeeName	Salary	DepartmentName	City	Designation	Email	mobile
102	Alice Johnson	72000	Finance	Bengaluru	Accountant	alice.johnson@example.com	9876543211
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Names ending with 'n'

```

137 • select * from employee1 where employeeName like '%n';
138
139

```

Result Grid

EmployeeID	EmployeeName	Salary	DepartmentName	City	Designation	Email	mobile
102	Alice Johnson	72000	Finance	Bengaluru	Accountant	alice.johnson@example.com	9876543211
103	Michael Brown	50000	IT	Hyderabad	Software Engineer	michael.brown@example.com	9876543212
105	David Wilson	70000	Marketing	Mumbai	Marketing Executive	david.wilson@example.com	9876543214
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Names containing 'ar'

Assignment 23 – IS NOT NULL

Scenario:

A company wants employees who have entered their mobile numbers.

Task:

Write an SQL query using **IS NOT NULL**.

```
142 • SELECT * FROM Employee1
143 WHERE Email IS not NULL;
144
```

EmployeeID	EmployeeName	Salary	DepartmentName	City	Designation	Email	mobile
101	John Smith	65000	HR	Chennai	HR Executive	john.smith@example.com	9876543
102	Alice Johnson	72000	Finance	Bengaluru	Accountant	alice.johnson@example.com	9876543
103	Michael Brown	85000	IT	Hyderabad	Software Engineer	michael.brown@example.com	9876543
104	Sophia Davis	95000	IT	Pune	Senior Software Engineer	sophia.davis@example.com	9876543
105	David Wilson	70000	Marketing	Mumbai	Marketing Executive	david.wilson@example.com	9876543

Assignment 24 – Aliasing (AS)

Scenario:

Your manager wants report headings to be more readable.

Task:

Write SQL queries to rename:

- EmployeeName → Employee Name
- Salary → Monthly Salary
- Department → Department Name

using **AS**.

```
145 • SELECT EmployeeName AS 'Employee Name', Salary AS 'Monthly Salary', DepartmentName AS 'Department Name'
146 FROM Employee1;
147
148
149
```

Employee Name	Monthly Salary	Department Name
John Smith	65000	HR
Alice Johnson	72000	Finance
Michael Brown	85000	IT
Sophia Davis	95000	IT

Assignment 25 – Mini Database Project

Scenario:

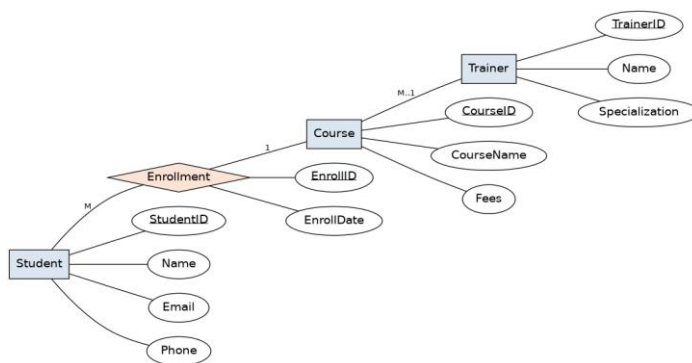
A coaching institute wants to maintain student information in MySQL.

The institute stores:

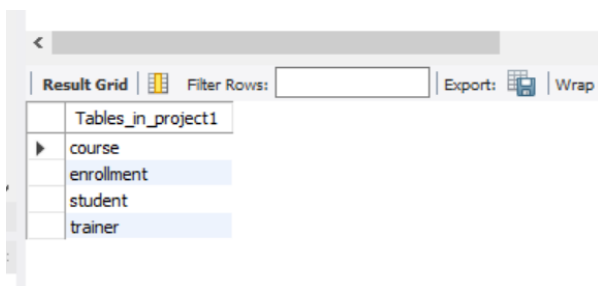
- Students
- Courses
- Trainers
- Enrollments

Task:

1. Design an ER Diagram.



2. Create the required tables.



3. Identify Primary Keys and Foreign Keys.
4. Normalize the tables up to **3NF**.
5. Insert at least **10 records** into each table.
6. Write SQL queries to:
 - Display all students.

```
208 • select * from student;
209
```

Result Grid

	StudentID	StudentName	Email	Fees
▶	1	Sathish Kumar	sathish.k@example.com	30000.00
	2	Divya Bharathi	NULL	28000.00
	3	Naveen Raj	naveen.r@example.com	25000.00
	4	Sowmya Lakshmi	sowmya.l@example.com	40000.00
	5	Ashwin Prasad	ashwin.p@example.com	20000.00
	6	Sneha Reddy	NULL	35000.00

student 46 x

- Display students enrolled in the Java course.

```
209 • SELECT s.StudentID, s.StudentName
210 FROM Student s
211 JOIN Enrollment e ON s.StudentID = e.StudentID
212 JOIN Course c ON e.CourseID = c.CourseID
213 WHERE c.CourseName = 'Java';
214
```

Result Grid

	StudentID	StudentName
▶	1	Sathish Kumar

- Display unique course names.

```
214 • select distinct coursename from course;
```

Result Grid

	coursename
▶	Java
	Python
	Web Development
	Data Science
	MySQL
	Advanced Java

- Sort students by name.

215 • `select * from student order by studentname asc;`

Result Grid

	StudentID	StudentName	Email	Fees
▶	5	Ashwin Prasad	ashwin.p@example.com	20000.00
	2	Divya Bharathi	NULL	28000.00
	3	Naveen Raj	naveen.r@example.com	25000.00
	9	Praveen Anand	praveen.a@example.com	22000.00
	10	Sandhya Rani	sandhya.r@example.com	45000.00
	1	Sathish Kumar	sathish.k@example.com	30000.00

- Display students whose fees are between ₹20,000 and ₹50,000.

216 • `select studentname, fees from student where fees between 20000 and 50000;`

Result Grid

	studentname	fees
▶	Sathish Kumar	30000.00
	Divya Bharathi	28000.00
	Naveen Raj	25000.00
	Sowmya Lakshmi	40000.00
	Ashwin Prasad	20000.00
	Sneha Reddy	35000.00

- Display students whose names start with S.

217 • `select studentname from student where studentname like 'S%';`

Result Grid

	studentname
▶	Sathish Kumar
	Sowmya Lakshmi
	Sneha Reddy
	Shalini Menon
	Sandhya Rani

- Display students with no email address.

218 • `select studentname, Email from student where email is NULL;`

Result Grid

	studentname	Email
▶	Divya Bharathi	NULL
	Sneha Reddy	NULL

- Rename StudentName as **Student Name** using AS.

219 • `select studentname as 'Student Name' from student;`

<   Filter Rows: Export:  Wrap Cell Content:

	Student Name
▶	Sathish Kumar
	Divya Bharathi
	Naveen Raj
	Sowmya Lakshmi
	Ashwin Prasad
	Aravind Reddy

7. Add comments to your SQL script explaining each section.

Added check script FYR.



Assignment
1_Supriya.sql